

Electrónica Digital Práctica

2025

Temario

1- Máquinas de estados finitos

2- Simulación II

Máquinas de estados Finitos

En la **estructura general** hay que completar 4 partes de código:

- Definición y asignación de estados, según el número de estados utilizaremos más o menos bits.
- Definición de registros para almacenar el estado actual y el siguiente. Deben ser del mismo tamaño en bits que el utilizado en el punto anterior
- Proceso de cambio de estado: Siempre es el mismo código
- Proceso de cálculo de siguiente estado y salida: Hay que rellenar el código correspondiente las transiciones del diagrama de estados

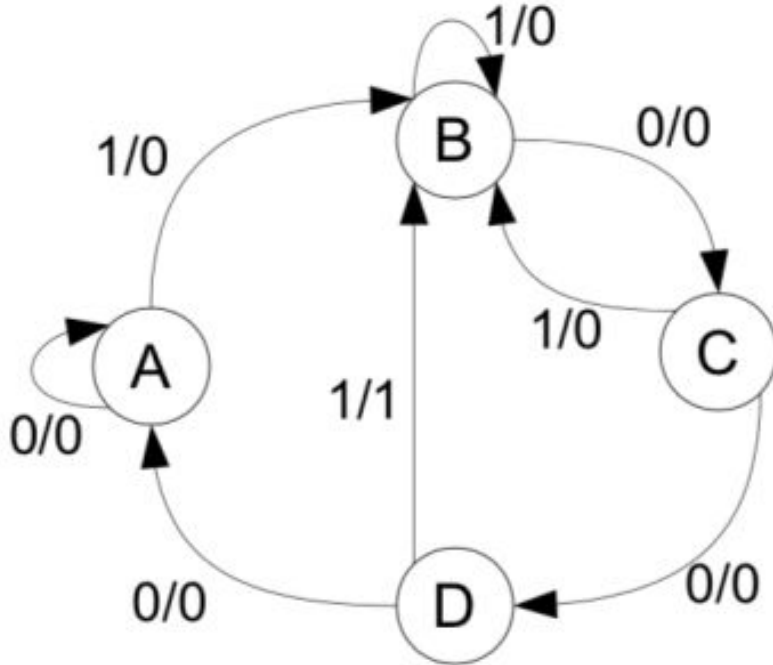
Máquinas de estados Finitos

En general utilizaremos una estructura del código en la que hay 2 procesos:

- Un proceso always que establece **cómo cambia de estado** (por flanco de subida, flanco de bajada, reset asíncrono, etc)
- Otro proceso always de **cálculo de los estados próximos y de salidas.**

```
module diag_estados(  
    input lista_entradas,  
    output reg lista_salidas);  
  
    // def y asignacion de estados  
    parameter lista_estados  
  
    // estado actual y siguiente  
    reg [N:0] current_state, next_state;  
  
    // proceso de cambio de estado  
    always @(posedge clk or posedge reset)  
        .....  
  
    // proceso de estado siguiente y salidas  
    always @(current_state, lista_entradas)  
        .....  
endmodule
```

Máquinas de estados Finitos - Ejemplo



```
module maquina_estados(  
    input x, clk, reset,  
    output reg z);
```

```
parameter A = 2'b00, B = 2'b01,  
          C = 2'b10, D = 2'b11;
```

asignacion
de estados

```
reg [1:0] current_state,next_state;
```

```
always @(posedge clk, posedge reset)  
begin  
    if(reset)  
        current_state <= A;  
    else  
        current_state <= next_state;  
end
```

asignac.
de
estado
siguiente

Continua ->

Máquinas de estados Finitos

- El proceso de cálculo del siguiente estado y salida se realiza con una única sentencia case
- La sentencia case debe contemplar todos los estados del diagrama de estados
- Antes de la sentencia case se recomienda establecer por defecto a cero todas las salidas.

Máquinas de estados Finitos

```
always @(current_state,x) begin
    z = 0;           // salida a 0
    case(current_state)
        A: if(x == 1) // estado A
            next_state = B;
        else
            next_state = A;
        B: if(x == 1) // estado B
            next_state = B;
        else
            next_state = C;
```

Continúa ->

```
C: if(x == 1) // estado C
    next_state = B;
else
    next_state = D;
D: if(x == 1) begin // estado D
    z = 1;
    next_state = B;
end
else
    next_state = A;
endcase
end
endmodule
```

Hands On !